1. Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

2. Load the Dataset

```
df = pd.read_csv("/content/creditcard.csv", on_bad_lines='skip')
```

3. View Dataset Information

```
df.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	
0	0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.11
1	0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.10
2	1	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.90
3	1	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.19
4	2	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.13

5 rows × 31 columns

```
df.tail()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22
99116	66970	1.383003	-1.100025	0.514116	-1.264196	-1.560304	-0.735428	-0.929887	-0.144582	-1.879341	...	-0.142905	-0.065429
99117	66971	-1.589012	0.449203	1.341944	0.152955	-0.071734	-0.711538	1.634046	-0.260976	-0.533781	...	0.042474	0.129532
99118	66972	0.990739	-0.322129	1.133347	0.640936	-0.212743	1.755410	-0.906331	0.629480	0.798532	...	0.037755	0.485242
99119	66972	-0.576857	1.215356	0.943351	-0.498420	0.879226	0.015361	0.954065	-0.272786	0.090732	...	-0.452060	-0.774950
99120	66973	-1.014594	-0.513014	1.924881	-1.623481	-0.235055	0.456966	-0.287827	0.313670	-0.683070	...	NaN	NaN

5 rows × 31 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99121 entries, 0 to 99120
Data columns (total 31 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Time    99121 non-null  int64
 1   V1      99121 non-null  float64
 2   V2      99121 non-null  float64
 3   V3      99121 non-null  float64
 4   V4      99121 non-null  float64
 5   V5      99121 non-null  float64
 6   V6      99121 non-null  float64
 7   V7      99121 non-null  float64
 8   V8      99121 non-null  float64
 9   V9      99121 non-null  float64
 10  V10     99121 non-null  float64
 11  V11     99121 non-null  float64
 12  V12     99121 non-null  float64
 13  V13     99121 non-null  float64
 14  V14     99121 non-null  float64
 15  V15     99121 non-null  float64
 16  V16     99121 non-null  float64
 17  V17     99121 non-null  float64
 18  V18     99121 non-null  float64
```

```
 19  V19     99121 non-null  float64
 20  V20     99121 non-null  float64
 21  V21     99120 non-null  float64
 22  V22     99120 non-null  float64
 23  V23     99120 non-null  float64
 24  V24     99120 non-null  float64
 25  V25     99120 non-null  float64
 26  V26     99120 non-null  float64
 27  V27     99120 non-null  float64
 28  V28     99120 non-null  float64
 29  Amount  99120 non-null  float64
 30  Class   99120 non-null  float64
dtypes: float64(30), int64(1)
memory usage: 23.4 MB
```

```
df.shape
```

```
(99121, 31)
```

Double-click (or enter) to edit

```
df.describe()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8
count	99121.000000	99121.000000	99121.000000	99121.000000	99121.000000	99121.000000	99121.000000	99121.000000	99121.000000
mean	42213.815861	-0.262004	-0.033117	0.676050	0.162447	-0.279002	0.094272	-0.108333	0.056136
std	16959.616226	1.860400	1.658620	1.325288	1.350168	1.356758	1.301340	1.212608	1.207004
min	0.000000	-56.407510	-72.715728	-33.680984	-5.172595	-42.147898	-26.160506	-31.764946	-73.216718
25%	33380.000000	-1.027973	-0.599728	0.177007	-0.712131	-0.898502	-0.647594	-0.600418	-0.137605
50%	44112.000000	-0.260205	0.077775	0.754452	0.191743	-0.314473	-0.157188	-0.069316	0.073908
75%	55528.000000	1.153321	0.735516	1.376924	1.032558	0.249811	0.486727	0.415134	0.359821
max	66973.000000	1.960497	18.902453	4.226108	16.715537	34.801666	22.529298	36.677268	20.007208

8 rows × 31 columns

4. Check Missing Values

```
df.isnull().sum()
```

	0
Time	0
V1	0
V2	0
V3	0
V4	0
V5	0
V6	0
V7	0
V8	0
V9	0
V10	0
V11	0
V12	0
V13	0
V14	0
V15	0
V16	0
V17	0
V18	0
V19	0
V20	0
V21	1
V22	1
V23	1
V24	1
V25	1
V26	1
V27	1
V28	1
Amount	1
Class	1

dtype: int64

5. Check Class Imbalance

```
df['Class'].value_counts()
```

Class	count
0.0	98898
1.0	222

dtype: int64

6. Feature Scaling

```
scaler = StandardScaler()
```

```
df['Amount'] = scaler.fit_transform(df[['Amount']])
df['Time'] = scaler.fit_transform(df[['Time']])
```

7: Separate Features and Target

```
X = df.drop('Class', axis=1)
y = df['Class']
```

8. Split into Training and Testing Data

```
df_cleaned = df.dropna(subset=['Class'])
X = df_cleaned.drop('Class', axis=1)
y = df_cleaned['Class']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (79296, 30)
Testing Data: (19824, 30)
```

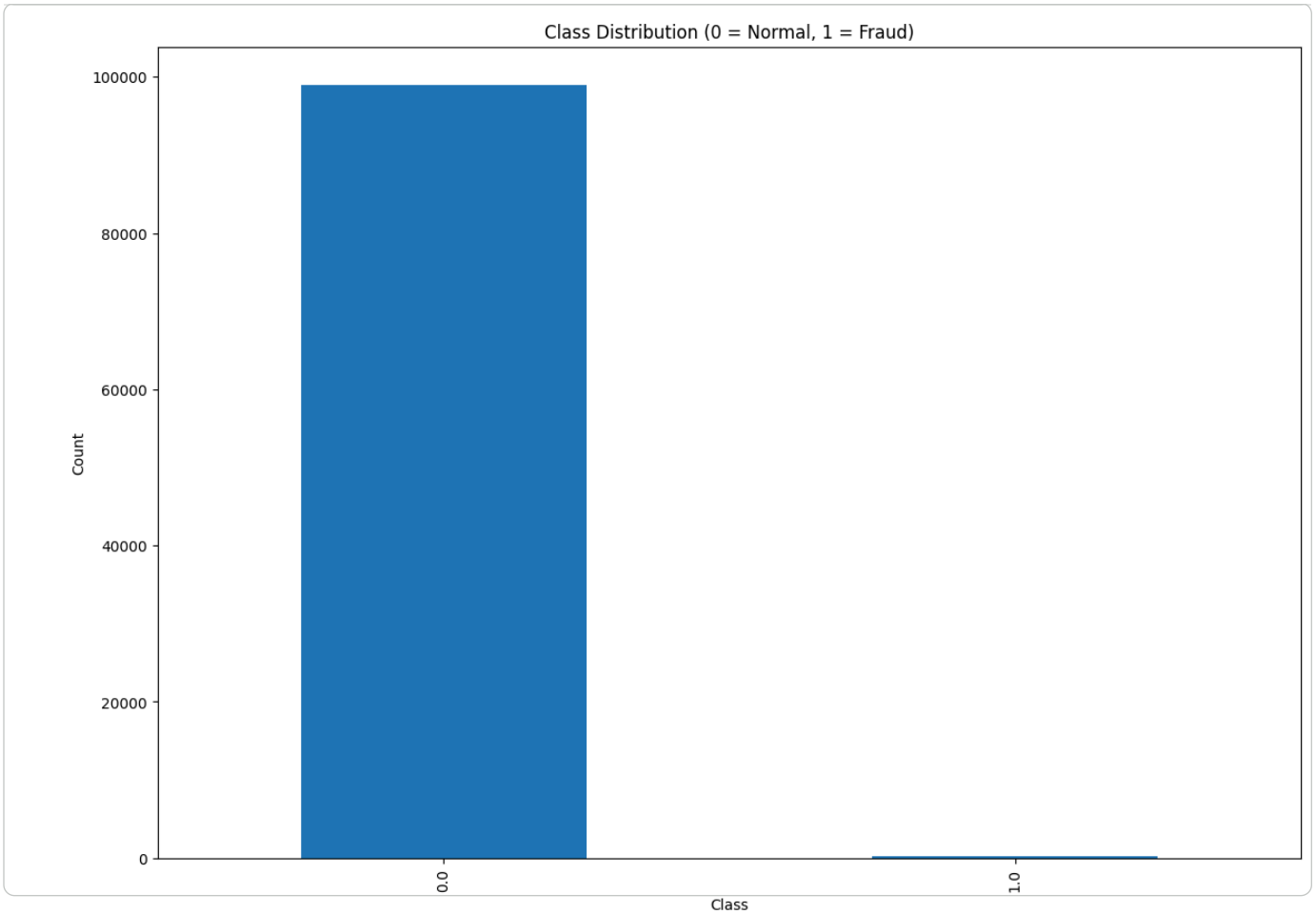
9. Handle Imbalanced Data using SMOTE

```
from imblearn.over_sampling import SMOTE

object_cols = X_train.select_dtypes(include='object').columns

for col in object_cols:
    X_train[col] = pd.to_numeric(X_train[col], errors='coerce')

combined_train_df = pd.concat([X_train, y_train], axis=1)
combined_train_df = combined_train_df.dropna()

X_train_cleaned = combined_train_df.drop(columns=[y_train.name])
y_train_cleaned = combined_train_df[y_train.name]

smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train_cleaned, y_train_cleaned)

print(y_train_res.value_counts())
```

```
Class
0.0    79118
1.0    79118
Name: count, dtype: int64
```

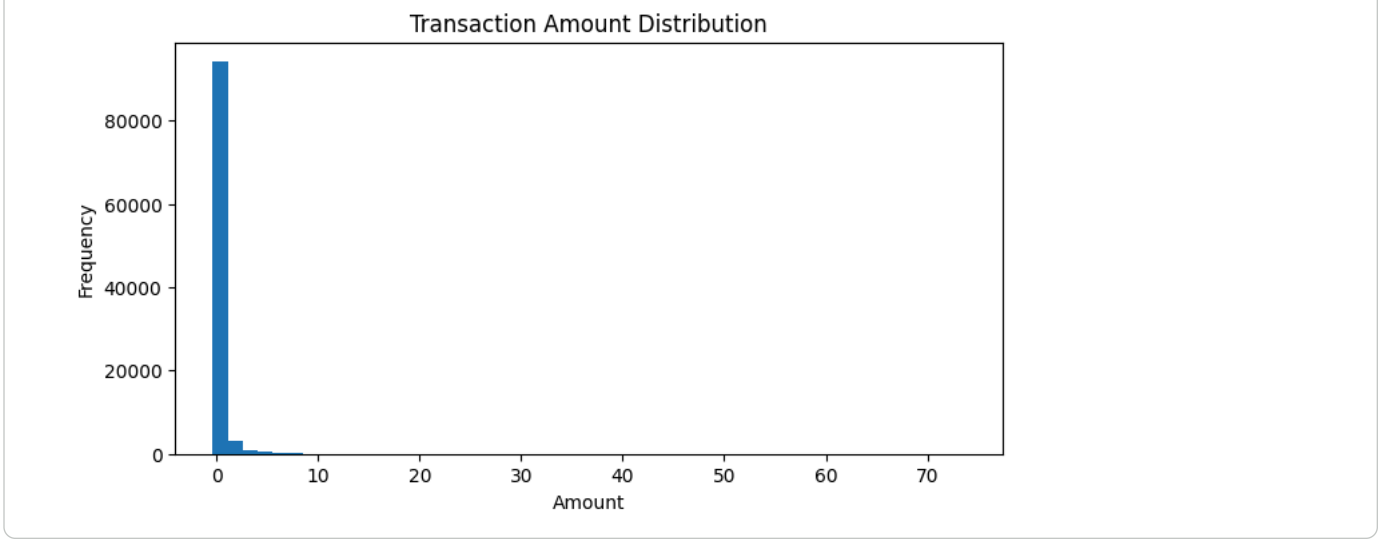
10. Class Distribution (Fraud vs Normal)

```
plt.figure(figsize=(14,10))
df['Class'].value_counts().plot(kind='bar')
plt.title("Class Distribution (0 = Normal, 1 = Fraud)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```



11. Distribution of Transaction Amount

```
plt.figure(figsize=(8,4))
plt.hist(df['Amount'], bins=50)
plt.title("Transaction Amount Distribution")
plt.xlabel("Amount")
plt.ylabel("Frequency")
plt.show()
```



12. Heatmap

```
plt.figure(figsize=(14,10))
correlation = df.corr()
plt.imshow(correlation)
```

```
plt.title("Correlation Heatmap")
plt.colorbar()
plt.show()
```